Day 1: The LLM inference stack: from prompt to tokens per second

LLM Inference 90-Day • 2026-09-09

Objectives

By the end of today you can:

- Trace the full path of a prompt: tokenization, the forward pass, sampling, and detokenization.
- Define **tokens per second** and measure it yourself in a notebook.
- Compute the **KV-cache size** for a model from its config with a worked example.
- Explain, in infra terms, why decoding is memory-bandwidth bound.

The LLM inference stack

A prompt travels through four stages:

Stage	What happens	Typical cost
Tokenize	Text → integer token IDs via a BPE vocabulary	~µs, CPU
Prefill	One forward pass over the whole prompt; builds the KV cache	Compute-bound
Decode	Generate one token at a time; each step attends to the KV cache	Memory-bandwidth bound
Detokenize	Token IDs → text	~µs, CPU

Everything expensive lives in decode. A backend engineer already knows the shape of this problem: a hot loop where each iteration reads a large buffer (the KV cache) and does a small amount of math. It is a memory-bandwidth roofline, the same regime as serving a large static asset with a tiny per-byte transform.

Throughput: tokens per second

The metric the whole field optimizes is output tokens per second:

```
throughput = new_tokens / wall_time_seconds
```

On a free Colab CPU with GPT-2 you will measure roughly **5–15 tokens/sec**. The same model on a T4 GPU lands around **40–80 tokens/sec**, and the improvement comes almost entirely from higher

memory bandwidth, not more FLOPs. Remember that ratio — it recurs for the entire program.

Worked example: KV-cache size for GPT-2

Every decoded token leaves behind its key and value vectors in every layer. The cache size in bytes is:

```
kv_cache_bytes = 2 × layers × seq_len × hidden × bytes_per_param
```

Plug in GPT-2's config: 12 layers, hidden size 768, fp32 (4 bytes), and a 2048-token context:

- Per token: `2 × 12 × 768 × 4` = **73,728 bytes** (~72 KiB)
- Full 2048-token context: `73,728 × 2048` = **150,994,944 bytes** ≈ **144 MB**

Two observations. First, the cache grows **linearly** with sequence length — long contexts are a memory problem before they are a compute problem. Second, the weights of GPT-2 (117M params in fp32) are ~468 MB, so a long context's cache is a real fraction of model size. You compute this live in the notebook.

Lab: day01-llm-inference-stack

Open `notebooks/day01-llm-inference-stack.ipynb` in Colab (badge at the top of the notebook) and run it top to bottom, ~30 minutes.

1. **Environment check** — confirm Python, torch, and whether a GPU is present. Expected: `cuda available: False` on CPU runtimes, `True` on a T4. 2. **Install** — `pip install transformers torch --quiet`. 3. **Tokenize** — load `gpt2`, tokenize "The future of AI is" and print the token IDs. Expected IDs: `[464, 3187, 286, 9552, 318]`. 4. **Generate** — call `model.generate` with `max_new_tokens=20` and print the decoded text. 5. **Measure** — wrap generation in `time.perf_counter()` and compute tokens/sec. Expected on CPU: roughly 5–15 tok/s. 6. **KV-cache math** — compute `2 × 12 × seq_len × 768 × 4` for seq_len 1024 and 2048 and print both. Expected: **72.0 MiB** and **144.0 MiB**.

If anything differs by an order of magnitude, stop and re-run — the lab is self-checking.

Checkpoints

1. Why is the decode phase memory-bandwidth bound rather than compute-bound? 2. A 24-layer, 1024-hidden model in fp16 serves a 4096-token context. What is its KV-cache size? 3. You double the batch size and tokens/sec per request drops. Is total throughput up or down? Why? 4. What does `max_new_tokens` control, and what happens if you set it larger than the model's context window?

Answers: 1. Each decode step reads the entire KV cache to do a tiny matrix-vector multiply — bytes moved dominate FLOPs. 2. `2 × 24 × 4096 × 1024 × 2` = 402,653,184 bytes ≈ **384 MB**. 3. Total throughput usually rises: the same memory reads are amortized over more tokens per step. 4. It caps how many new tokens generation may emit; exceeding the context window either errors or silently

truncates depending on the backend.

Readings

- Hugging Face, "How to generate text" — focus on the greedy/beam/sampling diagram; it is the decode loop in one page.
- Lilian Weng, "Large Transformer Model Inference Optimization" — read sections 1–2 for the memory-bandwidth argument.
- GPT-2 paper (Radford et al.) — skim the model table to see where 12 layers / 768 hidden come from.

Tomorrow

Day 2 zooms into the attention mechanism itself — why the KV cache exists at all — and you will profile per-layer latency to see the decode cost in numbers.